@TestFactory : Dynamic Tests in JUnit5

<u>@TestFactory:</u>

- It is not a Test method.
- It is used to create Test dynamically.

<u>Normal Test method:</u>

@Test : Its static, means, Test cases are known at compile time.

```
@Test
void addTest() {
    assertEquals(7, 4 + 3);
}
```

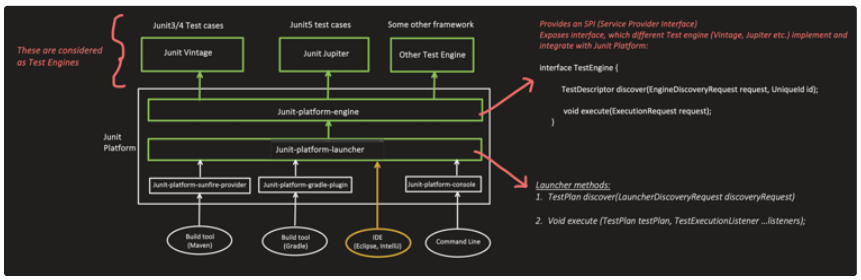
<u>Dynamic Test :</u>

@TestFactory: Means its not a normal test method, it generates Test case dynamically during runtime.

```
@TestFactory
DynamicTest addDynamicTest() {

        return DynamicTest.dynamicTest(
                        "addDynamicTest",
                       () -> assertEquals(7, 4+34)
         );
}
/*
addDynamicTest : This is display name
() -> assertEquals(7, 4+34) : This lambda expression is our executable,
our actual test case.
*/
```

Concept && Coding

<u>Internally what happens:</u>

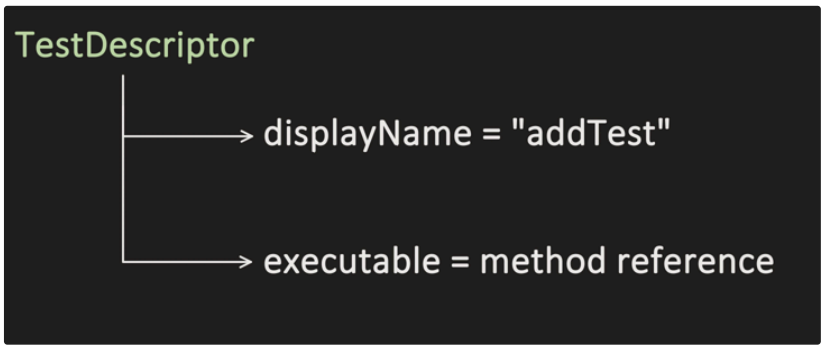


<u>From previous Architecture topic:</u>

we know that there are 2 phases:

1. Discover : TestDescriptor object is created
2. Execute : used TestDescriptor to execute the test cases.

Normal Tests (@Test):



TestMethodTestDescriptor **(Framework)**

```
@Override
public JupiterEngineExecutionContext execute(JupiterEngineExecutionContext context
                                             DynamicTestExecutor dynamicTestExecutor) {
    ThrowableCollector throwableCollector = context.getThrowableCollector();

    // @formatter:off
    invokeBeforeEachCallbacks(context);
        if (throwableCollector.isEmpty()) {
            invokeBeforeEachMethods(context);
            if (throwableCollector.isEmpty()) {
                invokeBeforeTestExecutionCallbacks(context);
                if (throwableCollector.isEmpty()) {
                    invokeTestMethod(context, dynamicTestExecutor);
                }
                invokeAfterTestExecutionCallbacks(context);
            }
            invokeAfterEachMethods(context);
        }
    invokeAfterEachCallbacks(context);
    // @formatter:on

    return context;
}
```

Lifecycle methods

Invoke Test method

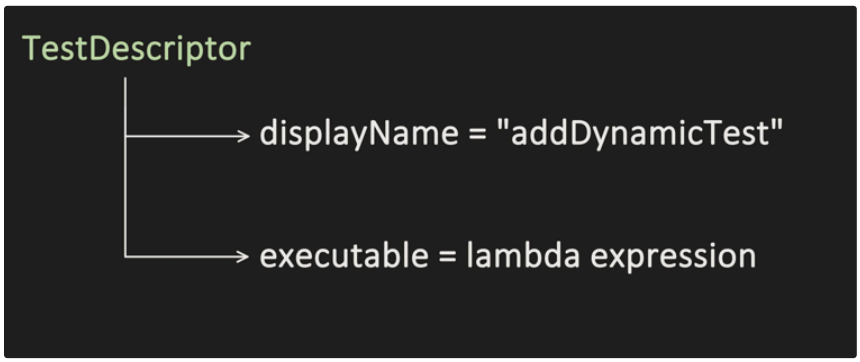
Lifecycle methods

Concept && Coding

```
protected void invokeTestMethod(JupiterEngineExecutionContext context, DynamicTestExecutor dynamicTestExecutor) {
    ExtensionContext extensionContext = context.getExtensionContext();
    ThrowableCollector throwableCollector = context.getThrowableCollector();

    throwableCollector.execute(() -> {
        try {
            Method testMethod = getTestMethod();
            Object instance = extensionContext.getRequiredTestInstance();
            executableInvoker.invoke(testMethod, instance, extensionContext, context.getExtensionRegistry(),
                    interceptorCall);
        }
        catch (Throwable throwable) {
            UnrecoverableExceptions.rethrowIfUnrecoverable(throwable);
            invokeTestExecutionExceptionHandlers(context.getExtensionRegistry(), extensionContext, throwable);
        }
    });
}
```

This is our executable, in which we are using reflection to invoke the test method.

Dynamic Test (@TestFactory):




```java
@Override
public JupiterEngineExecutionContext execute(JupiterEngineExecutionContext context,
                                             DynamicTestExecutor dynamicTestExecutor) {
    InvocationInterceptor.Invocation<Void> invocation = () -> {
        dynamicTest.getExecutable().execute();
        return null;
    };
    DynamicTestInvocationContext dynamicTestInvocationContext = new DefaultDynamicTestInvocationContext(
            dynamicTest.getExecutable());
    ExtensionContext extensionContext = context.getExtensionContext();
    ExtensionRegistry extensionRegistry = context.getExtensionRegistry();
    interceptorChain.invoke(invocation, extensionRegistry, InterceptorCall.ofVoid(
            (interceptor, wrappedInvocation) -> interceptor.interceptDynamicTest(wrappedInvocation,
                    dynamicTestInvocationContext, extensionContext)));
    return context;
}
```

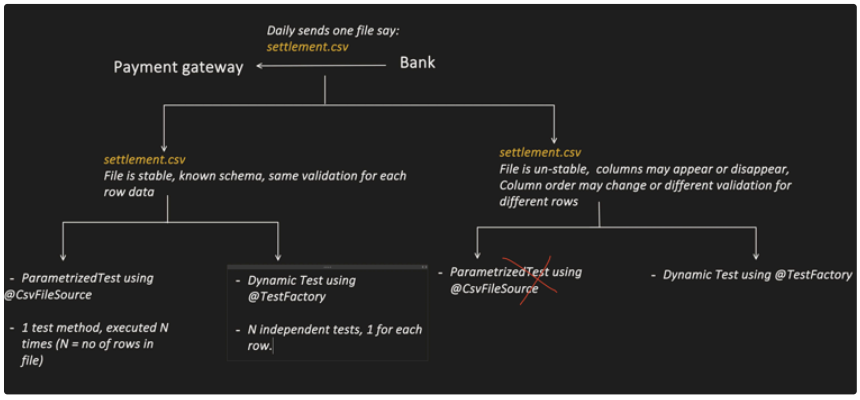
Uses Lambda expression as executable

Concept && Coding

Note:

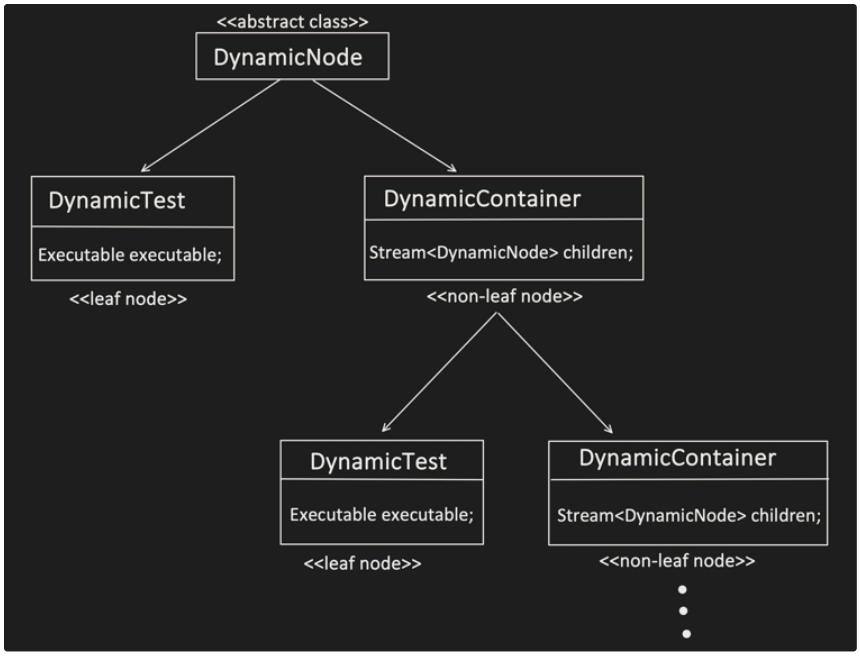
- *There is no @BeforeEach, @AfterEach lifecycle methods support in Dynamic Tests.*

Now, before we proceed further with Dynamic Test, we need to understand the Use case, where Dynamic test are really helpful:



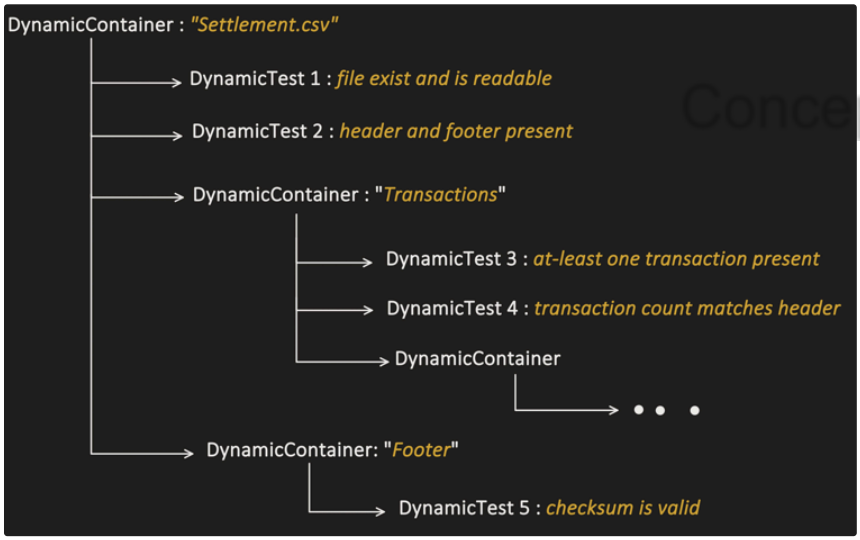
Above is just one example, there could be many other use cases.

Let understand, Dynamic test hierarchy:



Note: This actually follows composite design pattern in Low level design

DynamicContainer is just a logical grouping of dynamic tests



Concept && Coding

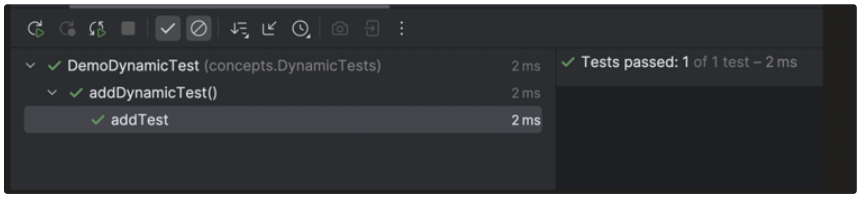
With this understanding, lets write the Dynamic tests:

1. **Single Dynamic Test**

```
//single test
@TestFactory
DynamicTest addDynamicTest() {

    return DynamicTest.dynamicTest(
            displayName: "addTest",
            () -> assertEquals(expected: 7, actual: 4+3)
    );
}

```

```java
// OR : Even DynamicNode can be used as return type, as it is parent of DynamicTest.

//single test
@TestFactory
DynamicNode addDynamicTestDynamicNode() {

    return DynamicTest.dynamicTest(
            displayName: "addTest",
            () -> assertEquals(expected: 7, actual: 4+3)
    );
}


```

Output:

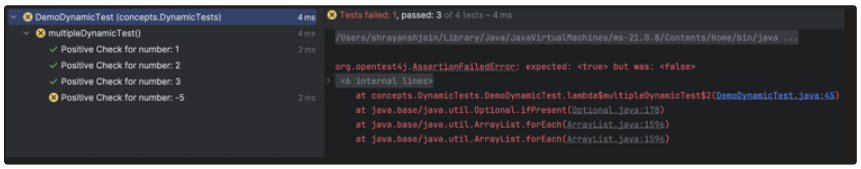


2. **Multiple Dynamic Test**

```java
@TestFactory
Stream<DynamicTest> multipleDynamicTest() {

    return Stream.of(values: 1, 2, 3, -5)
            .map((Integer n) -> {
                return DynamicTest.dynamicTest(
                        displayName: "Positive Check for number: " + n,
                        () -> assertTrue(condition: n > 0)
                );
            });
}

```

Concept && Coding

Output:



Apart from Stream<DynamicTest>, we can also have:

- Collection<DynamicTest>
- DynamicTest[]
- Iterator<DynamicTest>

```java
//COLLECTION

@TestFactory
```

```java
Collection<DynamicTest> multipleDynamicTestsCollectionReturn() {

    List<DynamicTest> tests = new ArrayList<>();

    for (int n : List.of(1, 2, 3, -5)) {
        tests.add(
                DynamicTest.dynamicTest(
                        displayName: "Positive Check for number: " + n,
                        () -> assertTrue(condition: n > 0)
                )
        );
    }

    return tests;
}

//ARRAY
@TestFactory
DynamicTest[] multipleDynamicTestsArrayReturn() {

    return new DynamicTest[] {
            DynamicTest.dynamicTest(
                    displayName: "Positive Check for number: 1",
                    () -> assertTrue(condition: 1 > 0)
            ),
            DynamicTest.dynamicTest(
                    displayName: "Positive Check for number: 2",
                    () -> assertTrue(condition: 2 > 0)
            ),
            DynamicTest.dynamicTest(
                    displayName: "Positive Check for number: 3",
                    () -> assertTrue(condition: 3 > 0)
            )
    };
}

```

Concept && Coding

3. **Dynamic Container : Different no of dynamic test for different rows**

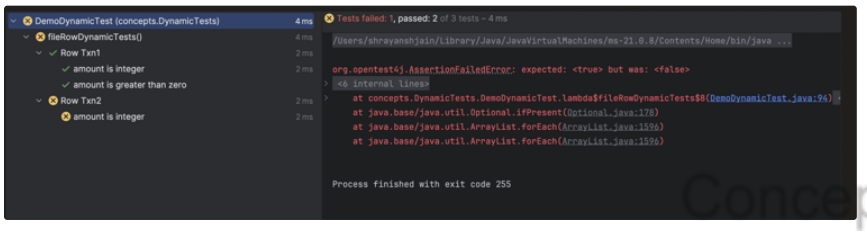
Sample file:

Txn_Id	Amount	Currency
Txn1	100	INR
Txn2	ABC	USD

```java
@TestFactory
Stream<DynamicContainer> fileRowDynamicTests() {

    List<String[]> rows = List.of(
            new String[]{"Txn1", "100", "INR"},
            new String[]{"Txn2", "ABC", "USD"}
    );

    //Iterates over each row
    return rows.stream().map((String[] row) -> {

        String txnId = row[0];
        String amount = row[1];
        String currency = row[2];

        List<DynamicTest> tests = new ArrayList<>();

        //For each row, this dynamic test is created
```

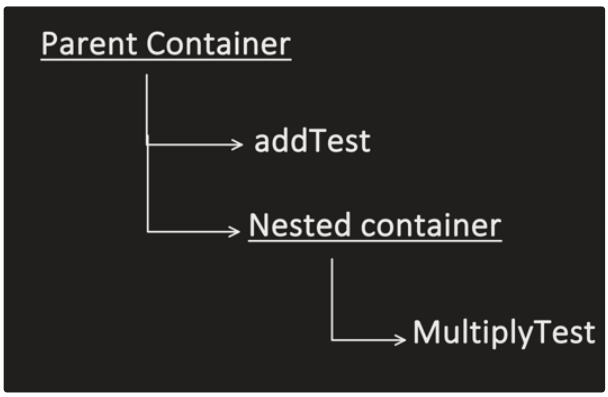
```java
        tests.add(DynamicTest.dynamicTest(
                displayName: "amount is integer",
                () -> assertTrue(amount.matches(regex: "\\d+"))
        ));

        //This dynamic test created for specific rows only
        if ("INR".equals(currency)) {
            tests.add(DynamicTest.dynamicTest(
                    displayName: "amount is greater than zero",
                    () -> assertTrue(condition: Integer.parseInt(amount) > 0)
            ));
        }

       //Now, all the dynamic tests created for 1 row, need to be grouped, so used Dynamic Container
        return DynamicContainer.dynamicContainer(
                displayName: "Row " + txnId,
                tests
        );
    });
}

```

Output:



Concept && Coding

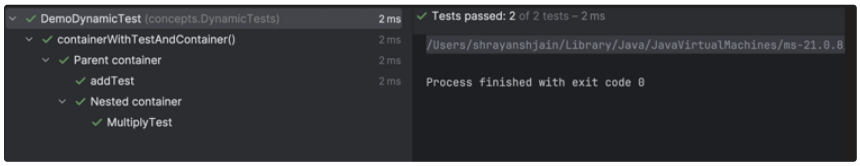
4. **Dynamic Container → Dynamic Test + Dynamic Container**




```java
@TestFactory
DynamicContainer containerWithTestAndContainer() {

    // A simple test directly inside the parent container
    DynamicTest simpleTest = DynamicTest.dynamicTest(
            displayName: "addTest",
            () -> assertEquals(expected: 2, actual: 1 + 1)
    );

    // A nested container with its own test
```

```java
    DynamicContainer nestedContainer = DynamicContainer.dynamicContainer(
            displayName: "Nested container",
            Stream.of(
                    DynamicTest.dynamicTest(
                            displayName: "MultiplyTest",
                            () -> assertEquals(expected: 4, actual: 2 * 2)
                    )
            )
    );

    // Parent container containing both
    return DynamicContainer.dynamicContainer(
            displayName: "Parent container",
            Stream.of(
                    simpleTest,
                    nestedContainer
            )
    );
}

```

Output:



Concept && Coding